RIS Lab II
Report for Assignment 1

Yiheng Cao, Eduard-Antonio Zippenfenig, Wai Huen Sin, Amanuel Basaznew Legesse

***Abstract*— In this assignment we explored the basic interfaces and hardware capabilities of the duckiebot *teal*. The associated source code for Task 2 is available in the attached zip file.**

TASK 1.1

All availabe Ros Topics of the duckiebot:

```
qwe@qwe-virtual-machine:~$ rostopic list
/diagnostics
/rosout
/rosout_agg
/teal/auto_calibration_calculation_node/car_cmd
/teal/auto_calibration_node/car_cmd
/teal/button_driver_node/event
/teal/camera_node/camera_info
/teal/camera_node/image/compressed
/teal/car_cmd_switch_node/cmd
/teal/client_count
/teal/connected_clients
/teal/coordinator_node/car_cmd
/teal/diagnostics/code/profiling
/teal/diagnostics/ros/links
/teal/diagnostics/ros/node
/teal/diagnostics/ros/parameters
/teal/diagnostics/ros/topics
/teal/display_driver_node/fragments
/teal/duckiebot_il_lane_following/car_cmd
/teal/front_center_tof_driver_node/range
/teal/fsm_node/mode
/teal/imu_node/data
/teal/imu_node/temperature
/teal/joint_states
/teal/joy
/teal/joy/set_feedback
```

Fig. 1. Ros Topics part of duckie bot 1

```
/teal/joy_mapper_node/car_cmd
/teal/joy_mapper_node/joystick_override
/teal/kinematics_node/velocity
/teal/lane_controller_node/car_cmd
/teal/lane_recovery_node/car_cmd
/teal/lane_supervisor_node/car_cmd
/teal/led_emitter_node/led_pattern
/teal/left_wheel_encoder_node/tick
/teal/right_wheel_encoder_node/tick
/teal/simple_stop_controller_node/car_cmd
/teal/velocity_to_pose_node/pose
/teal/wheels_driver_node/emergency_stop
/teal/wheels_driver_node/wheels_cmd
/teal/wheels_driver_node/wheels_cmd_executed
/tf
/tf_static
```

Fig. 2. Ros Topics part of Duckie Bot (continued)

Two chosen topics from above:
/teal/wheels_driver_node/wheels_cmd
message type: duckietown_msgs/WheelsCmdStamped
/teal/wheels_driver_node/wheels_cmd_executed
message type: duckietown_msgs/WheelsCmdStamped

TASK 1.2

We use command `rosrun plotjuggler plotjugglerto` open PlotJuggler. We used it to visualize the frontal range sensor data of the duckiebot. We visualize the message Robots received. The screenshot shows an obstacle moving away from the minimum to maximum range shown in Figure 3.

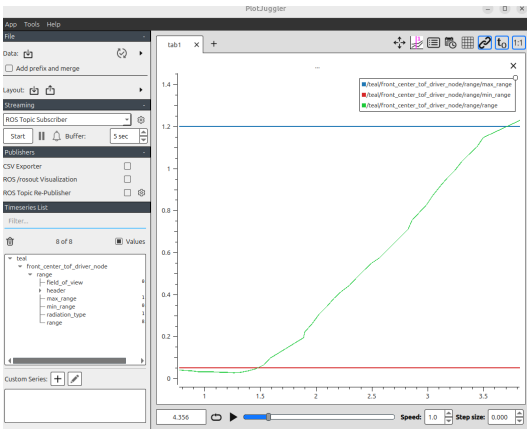

Fig. 3. Message from the sensor received in line mode

Afterwards, we changed the plot mode into scatter mode, which can show a better visual representation.

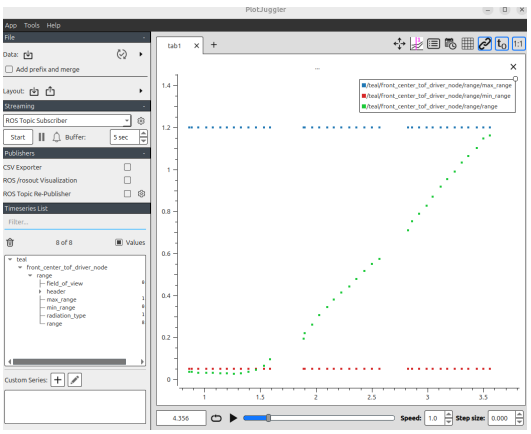

Fig. 4. Message from the sensor received in scatter mode

TASK 1.3

To complete this task, we first installed the software packages required for RViz visualization. This was done using the command

`sudo apt install ros-one-imu-tools ros-one-jsk-rviz-plugins`, which installs tools for IMU data visualization and additional RViz plugins. We also cloned the specified GitHub repository and checked out the required branch to ensure that we had all necessary files and configurations for the robot model and sensor data.

Next, we configured the RViz scene to resemble the reference visualization as closely as possible by adding and adjusting several display elements. In the Global Options settings, we set the Fixed Frame to `teal/footprint`, which defines the coordinate frame used as the reference for all visualized data. For the Grid display, we increased the cell count to 100 and reduced the cell size to 0.1. This produced a denser grid that provides a clearer sense of scale and positioning in the environment.

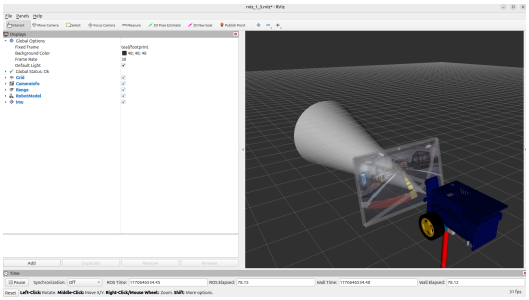

Fig. 5. RViz visualization of the robot set-up with the proper displays and the settings necessary

We then added the Robot Model display to visualize the robot itself using its URDF description. To display camera data, we configured the Camera Info display by selecting the topic `/teal/camera_node/camera_info`. We also adjusted the far clipping distance to 0.1 to control the rendering range. The image stream was set to the topic `/teal/camera_node/image` with transport hints set to `compressed` to reduce bandwidth usage. Because compressed images require decoding, we installed the necessary plugin using the command `sudo apt install ros-one-image-transport-plugins`, which enables RViz to decompress and display the image correctly.

For inertial data visualization, we added the IMU display and selected the topic `/teal/imu_node/data`. Acceleration visualization was enabled, and the size of the acceleration marker was increased to make motion changes easier to observe. Finally, we added a Range display to visualize distance measurements from the time-of-flight sensor, using the topic `/teal/front_center_tof_driver_node/range`. Together, these configurations allowed us to recreate a detailed and functional visualization of the robot and its sensor data within RViz.

Task 2

For the interface, the robot wheels are made to move through publishing to the the `/teal/wheels_driver_node/wheels_cmd` topic

The type of message that is published is a '`duckietown_msgs/WheelsCMdStamped`' and the message contains a header as well as floating numbers '`vel_left`' and '`vel_right`'. The former is used for timing and synchronization while the latter two are used in the cmd message object for the velocity of the left and right wheels of the Duckiebot.

After having created the `drive_forward_pkg`' the `driver_forward_node.cpp` is made as well as an updated `CMakeLists` that contains the executable as well as the linked libraries and dependencies.

By first creating a publisher to the wheels of the node through:

```
ros::Publisher pub = nh.advertise<duckietown_msgs::WheelsCmdStamped>(
    "/teal/wheels_driver_node/wheels_cmd", 1);
```

and then setting the velocities of both wheels a while loop is used for movement of 3 seconds by continuously publishing for this amount of time through the logic shown below:

```
while (ros::ok() && (ros::Time::now() - start_time).toSec() < 3.0)
{
    cmd.header.stamp = ros::Time::now();
    pub.publish(cmd);
    rate.sleep();
}
```

Once the loop is completed, the velocities of the wheels are set back to 0 and published to the node ensuring that it stops afterwards.

Contributions

- Yiheng Cao - present during the group work. Worked on and wrote about Task 1.2
- Eduard-Antonio Zippenfenig- present during the group work. Worked on and wrote about Task 1.3
- Wai Huen Sin- present during the group work. Worked on and wrote about Task 1.1Pic
- Amanuel Basaznew Legesse- present during the group work. Worked on and wrote about Task 2